



Ejercicios de Programación Dinámica

Ej. 1 — Considerar el alfabeto $\Sigma = \{a, b, c\}$. Sobre los elementos de Σ se define la siguiente tabla de multiplicación, en la que las filas indican el factor izquierdo y las columnas el derecho.

	a	b	c
a	b	b	a
b	c	b	a
c	a	c	c

Es decir, $a \cdot b = b$, $b \cdot a = c$, etc.

Nótese que la multiplicación definida así no es ni conmutativa ni asociativa. Diseñar un algoritmo eficiente que examine una expresión $x = x_1 x_2 \dots x_n$ de caracteres de Σ y decida si es posible o no poner paréntesis en x de forma que el valor resultante de evaluar la expresión sea a .

Ejemplo: Si

$$x = bbbba,$$

el algoritmo debe decir SI porque

$$(b(bb))(ba) = a.$$

Esta expresión no es única. Por ejemplo, también es válido

$$(b(b(b(ba)))) = a.$$

¿Qué coste tiene el algoritmo en términos de n ?

Ej. 2 — Una empresa de distribución de un determinado producto tiene dos almacenes A_1 y A_2 desde los cuales satisface los pedidos de sus clientes $C_j, 1 \leq j \leq n$. Denotamos por d_j el número de unidades de la demanda del cliente $C_j, 1 \leq j \leq n$, y por r_i el número de unidades de inventario en $A_i, 1 \leq i \leq 2$. Se supone que $r_1 + r_2 = d_1 + \dots + d_n$. Sea $c_{ij}(x_{ij})$ el coste de transportar x_{ij} unidades del producto desde el almacén A_i hasta el cliente C_j . El problema del almacén consiste en calcular los enteros no negativos $x_{ij}, 1 \leq i \leq 2, 1 \leq j \leq n$, tales que $x_{1j} + x_{2j} = d_j, 1 \leq j \leq n$ y que $\sum_{ij} c_{ij}(x_{ij})$ sea mínimo.

Sea $g_i(x)$ el coste incurrido por una solución óptima del problema de satisfacer los pedidos de los clientes $C_1, \dots, C_i$ suponiendo que A_1 tiene un inventario de x unidades (y, por tanto, el inventario de A_2 es $\sum_{1 \leq j \leq i} d_j - x$). Entonces, el coste de una solución óptima del problema del almacén es $g_n(r_1)$.

Utilizar el principio de optimalidad para encontrar una relación recurrente para los $g_i(x)$. Diseñar un algoritmo eficiente para resolver esa relación recurrente y obtener los valores óptimos de $x_{ij}, 1 \leq i \leq 2, 1 \leq j \leq n$.

Ej. 3 — Deben procesarse N trabajos. Hay disponibles dos máquinas, A y B . Si el trabajo i se procesa en la máquina A entonces se requieren a_i unidades de tiempo de procesamiento,

mientras que si se procesa en la máquina B se requieren b_i unidades. Debido a las peculiaridades de las máquinas y de los trabajos, es bastante posible que $a_i \geq b_i$ para algunos i mientras que $a_j < b_j$ para otros. Diseñar un algoritmo que calcule el tiempo mínimo necesario para procesar todos los trabajos. Nótese que los trabajos no pueden partirse entre las dos máquinas. Indicar también cómo calcular la asignación óptima de trabajos a las máquinas.

Ej. 4 — Sobre el río Ebro hay n estafetas de correos. En cada estafeta se puede alquilar un bote que permite ir a cualquier otra estafeta río abajo (es imposible remontar la corriente). La tarifa indica el coste del viaje de i a j para cualquier punto de partida i y cualquier punto de llegada j más abajo en el río. Puede suceder que un viaje de i a j sea más caro que una sucesión de viajes más cortos, en cuyo caso se tomaría un primer bote hasta una estafeta k y un segundo bote para continuar a partir de k . No hay coste adicional por cambiar de bote. Diseñar un algoritmo eficiente que determine el coste mínimo para ir de i a j .

Ej. 5 — Dada una secuencia $X = \langle x_1, x_2, \dots, x_n \rangle$, otra secuencia $Z = \langle z_1, z_2, \dots, z_k \rangle$ se dice subsecuencia de X si existe una secuencia creciente de índices $\langle i_1, i_2, \dots, i_k \rangle$ entre 1 y n tal que para todo $j = 1, 2, \dots, k$, se tiene que $x_{i_j} = z_j$. Por ejemplo, $Z = \langle B, C, D, B \rangle$ es una subsecuencia de $X = \langle A, B, C, B, D, A, B \rangle$ cuya correspondiente secuencia de índices es $\langle 2, 3, 5, 7 \rangle$

Dadas dos secuencias X e Y , decimos que una secuencia Z es una subsecuencia común de X e Y si Z es subsecuencia de ambas (X e Y). Por ejemplo, dadas $X = \langle A, B, C, B, D, A, B \rangle$ e $Y = \langle B, D, C, A, B, A \rangle$ la secuencia $\langle B, C, A \rangle$ es una subsecuencia común de ambas. No obstante, la secuencia $\langle B, C, A \rangle$ no es una subsecuencia común de máxima longitud (SCML) de X e Y , puesto que tiene longitud 3 y la subsecuencia $\langle B, C, B, A \rangle$ que es también común a X e Y , tiene longitud 4. La secuencia $\langle B, C, B, A \rangle$ es una SCML de X e Y , al igual que la secuencia $\langle B, D, A, B \rangle$ puesto que no existe ninguna subsecuencia común de X e Y de longitud 5.

Dadas dos secuencias $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$ el problema de la subsecuencia común de máxima longitud consiste en encontrar una subsecuencia común de X e Y que tenga longitud máxima.

Puede demostrarse fácilmente que una SCML de dos secuencias dadas, X e Y , contiene una SCML de sendos prefijos de X e Y . Apoyándose en ese hecho, diseñar un algoritmo de programación dinámica que resuelva el problema del cálculo de una SCML de dos secuencias dadas. Analizar la eficiencia de la solución propuesta.

Ej. 6 — Recordemos el problema del cambio en monedas. Se trata de pagar una cantidad, N , a un cliente usando el menor número posible de monedas. En clase hemos visto una solución voraz para este problema. Sin embargo, aunque esa solución es muy eficiente, funciona bien sólo en algunos casos. Con ciertos sistemas de monedas el algoritmo puede encontrar una solución subóptima o no encontrar solución aún cuando ésta exista. Diseñar un algoritmo que funcione bien para cualquier sistema de monedas.

Vamos a suponer que en nuestro sistema de monedas hay n tipos diferentes de monedas, $\{1, 2, \dots, n\}$, de forma que la moneda i vale d_i unidades ($d_i > 0$), $1 \leq i \leq n$ y que hay un número ilimitado de monedas de todos los tipos.

Ej. 7 — Se denomina *arbitraje* al uso de las discrepancias en las tasas de cambio de divisas para obtener un beneficio. Por ejemplo, durante un corto espacio de tiempo puede suceder que un euro valga 0'95 dólares, un dólar valga 0'75 libras esterlinas y una libra esterlina valga 1'45 euros. En una situación como esa, partiendo de un euro podemos obtener:

$$1 \text{ €} = 1 \cdot 0'95 (\text{\$/ €}) \cdot 0'75 (\text{£/ \$}) \cdot 1'45 (\text{£/ €}) = 1'033125 \text{ euros}$$

con un beneficio neto de algo más del 3'3%.

Supondremos que hay n tipos diferentes de moneda, y que se dispone de una tabla $C(1..n, 1..n)$ donde $C(i, j)$ es la tasa de cambio entre (una unidad de) la divisa i y la divisa j . Por ejemplo, si el euro es la divisa 1, y la libra esterlina la 2, entonces (según hemos dicho antes) $C(2, 1) = 1'45$.

Se cumple que $C(i, j) = 1/C(j, i)$ para todo i y j , y que $C(i, i) = 1$.

Diseñar un algoritmo de programación dinámica que dada la matriz C y el identificador i de una divisa, nos proporcione el valor del mejor arbitraje, donde este consiste en una secuencia $i_1, i_2, \dots, i_k$ de identificadores distintos entre si, y distintos de i , tal que su valor $C(i, i_1) \cdot C(i_1, i_2) \cdot \dots \cdot C(i_{k-1}, i_k) \cdot C(i_k, i)$ es máximo.

Ej. 8 — El principio de optimalidad **no** es cierto para todo problema cuya solución pueda verse como el resultado de una secuencia de decisiones. Encontrar dos problemas para los que el principio de optimalidad no sea válido y explicar por qué dicho principio no es válido para esos problemas.

Ej. 9 — Un ayuntamiento tiene un presupuesto P para construir jardines y J posibles emplazamientos de jardines. El coste de construir un jardín en el emplazamiento j es c_j para $1 \leq j \leq J$.

1. Diseñar un algoritmo que produzca como salida el mayor gasto que es posible realizar para construir jardines con el presupuesto P .
2. Diseñar otro algoritmo que proporcione la lista con los emplazamientos afortunados cuyo gasto total sea el proporcionado por el algoritmo anterior.
3. Calcular los costes temporales y espaciales de ambos algoritmos.
4. Suponiendo que quedan sólo D días para gastar el presupuesto, que en construir el jardín en el emplazamiento j se tarda d_j días y que sólo se pueden construir los jardines de forma secuencial, modificar el algoritmo del primer punto para que tenga en cuenta esta nueva restricción y calcular su coste.

Ej. 10 — Dados N objetos de longitud $l_1, l_2, \dots, l_N$, (naturales positivos) y un número natural L , se desea saber si existe algún subconjunto de los N objetos cuya suma de longitudes sea exactamente L .

1. Diseñar una ecuación en recurrencias que devuelva verdad si existe tal subconjunto y falso en caso contrario.
2. Diseñar un algoritmo iterativo para la ecuación anterior con un coste en espacio de $O(L)$. Calcular el coste en tiempo.

Modificar el algoritmo anterior para que adicionalmente se obtenga uno de esos subconjuntos de objetos (en caso de que exista alguno). Calcular el coste en tiempo y en espacio de la solución.

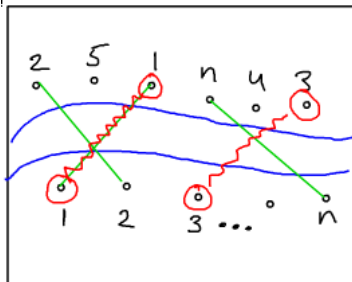
Ej. 11 — Dados n enteros, el problema de la partición es encontrar una partición en dos conjuntos que tengan la misma suma o determinar que tal partición no existe. Diseñar y analizar un algoritmo de programación dinámica para resolver el problema.

Ej. 12 — Vamos a realizar un viaje largo. Se empieza en el kilómetro 0. En el camino hay n hoteles, en los kilómetros $a_1 < a_2 < \dots < a_n$, cada a_i medido desde el punto inicial. Sólo podemos parar en esos hoteles, pero podemos elegir en cuáles parar. Hay que parar en el último, que es nuestro destino. Queríamos viajar 200 kilómetros cada día, pero puede no ser posible (según como se distribuyan los hoteles). Si viajamos x kilómetros en un día, la penalización para ese día será de $(200 - x)^2$. Queremos planear el viaje para minimizar la penalización total (esto es, la suma de las penalizaciones correspondientes a los días que dure el viaje). Proporcionar un algoritmo para resolver el problema.

Ej. 13 — Tenemos un conjunto de cajas rectangulares en tres dimensiones. La caja i tiene altura $h(i)$, anchura $w(i)$ y profundidad $d(i)$ (números reales). Queremos construir una pila de cajas tan alta como sea posible, pero sólo se puede apilar una caja sobre otra si cada una de las medidas de los lados de la base (bidimensional) de la caja de debajo son estrictamente mayores

que las de la caja que está encima. Es posible rotar las cajas para que cualquier cara actúe como base (podemos eliminar esta restricción si supone un problema).

Ej. 14 — Tenemos un mapa con un río que lo cruza por la mitad (horizontalmente). Hay n ciudades en la zona sur con coordenadas x definidas por $a(1), a(2), \dots, a(n)$ y n ciudades al norte con coordenadas x definidas por $b(1), b(2), \dots, b(n)$. Queremos conectar tantas ciudades entre sí mediante puentes de forma que los puentes no se crucen. Sólo es posible conectar la ciudad i del norte con la ciudad i del sur.



Podemos suponer que las ciudades del sur están ordenadas según su coordenada x y que son las del norte las que no tienen por qué estar ordenadas.

Ej. 15 — Tenemos una expresión lógica que consiste en una cadena de símbolos ‘true’, ‘false’, ‘and’, ‘or’ y ‘xor’. Contar el número de formas de poner paréntesis en la expresión de manera que el resultado se evalúe a ‘true’.

Ejemplo: Sólo hay una forma de agrupar ‘true and false xor true’ de forma que se evalúe a ‘true’.

Ej. 16 — Considerar un juego con n monedas cuyos valores son $v(1), \dots, v(n)$ (n par). Cada jugador, en su turno, elige la primera o la última moneda, la saca de la fila y obtiene el valor representado. Determinar el máximo valor de dinero que se podemos ganar si elegimos primero.

Ej. 17 — Debemos hacer un recorrido entre dos ciudades en un coche eléctrico, en el menor tiempo posible. El vehículo tiene una autonomía de x kilómetros después de recargarlo completamente (en las condiciones habituales de ese recorrido). El sistema de ayuda del vehículo dispone de la información sobre los cargadores, con las distancias entre los mismos a lo largo del recorrido. Teniendo en cuenta que el tiempo de carga no es cero, se desea hacer el menor número posible de paradas.

Ej. 18 — Evaluar y mejorar las páginas de la Wikipedia hispana sobre este tema (o alguno relacionado) (<http://es.wikipedia.org/>) (y el resultado final publicado allí), contrastándolo con la bibliografía y con las páginas equivalentes de la Wikipedia inglesa (<http://en.wikipedia.org/>). Naturalmente, esta última también es posible que pueda mejorarse.

Ej. 19 — Comprobar qué información nos ofrece alguna de las inteligencias artificiales disponibles, en español y en inglés, sobre:

- Programación dinámica (*dynamic programming*)

¿Proporciona respuestas? ¿Son correctas y adecuadas? ¿Se podrían mejorar?

Ej. 20 — Dada una lista $L[1..n]$ de n números enteros, queremos encontrar la longitud de la máxima sublista no decreciente (MSND), esto es, la longitud de la sublista más larga en la que sus elementos están en orden no decreciente.

Por ejemplo, para la lista $L = [3, 10, 2, 1, 20]$ las posibles sublistas no decrecientes son:

- Longitud 1: $[3], [10], [2], [1], [20]$

- Longitud 2: [3, 10], [3, 20], [10, 20], [2, 20], [1, 20]
- Longitud 3: [3, 10, 20]

No hay ninguna MSND de longitud mayor que 3.

Ejemplos:

- Entrada: [3, 10, 2, 1, 20]
Salida: 3
Explicación: la MSND es [3, 10, 20]
- Entrada: [3, 2]
Salida: 1
Explicación: hay dos MSND, y son [3] y [2]
- Entrada: [50, 3, 10, 7, 40, 80]
Salida: 4
Explicación: la MSND es [3, 7, 40, 80]

Puede demostrarse fácilmente que una MSND de una lista dada, L , contiene una MSND de un prefijo de L . Apoyándose en ese hecho:

1. Escribir la ecuación en recurrencias que representa la longitud de la MSND.
2. Diseñar un algoritmo de programación dinámica que resuelva el problema del cálculo de una MSND de una lista dada.
3. Analizar la eficiencia de la solución propuesta. No hace falta un análisis formal, basta con una idea genérica del coste.
4. Proponer una solución eficiente para resolver el problema.

Ej. 21 — Dada una vara que mide n centímetros y un vector que indica los precios de todas las varas de tamaño menor o igual que n , determinar cuál es el máximo valor que podemos conseguir cortando la vara y vendiendo los trozos obtenidos.

Ejemplos:

- Si la longitud es 8 y los precios son:

longitud	1	2	3	4	5	6	7	8
precio	1	5	8	9	10	17	17	20

El máximo valor que podemos obtener es de 22 (cortando la vara en dos piezas de longitud 2 y 6).

- Si la longitud es 8 y los precios son:

longitud	1	2	3	4	5	6	7	8
precio	3	5	8	9	10	17	17	20

El máximo valor que podemos obtener es de 24 (cortando la vara en 8 piezas de longitud 1).

Se pide:

- a) Si $corteVara(n)$ es el mejor beneficio que podemos obtener con una vara de longitud n , escribir la ecuación en recurrencias que permite calcular este valor.
- b) Diseñar un algoritmo que resuelva el problema del cálculo de dicho valor utilizando las ecuaciones recursivas.
- c) Analizar la eficiencia de la solución propuesta. No hace falta un análisis formal, basta con una idea genérica del coste en tiempo y espacio en función de n .
- d) Proponer una solución eficiente en tiempo para resolver el problema.

Ej. 22 — Dada una matriz triangular, encontrar el camino de suma mínima de arriba a abajo, desde la fila 1 y columna 1; en cada paso podemos movernos a las posiciones adyacentes de la fila siguiente, esto es, si estamos en la columna j de la fila i podemos ir a la columna j o la $j + 1$ de la fila $i + 1$.

Ejemplos:

M_1	1	2	3	4
1	2	0	0	0
2	3	7	0	0
3	8	5	6	0
4	6	1	9	3

M_2	1	2	3	4
1	3	0	0	0
2	6	9	0	0
3	8	7	1	0
4	9	6	8	2

Resultados:

Matriz M_1 : 11. El camino es $2 \rightarrow 3 \rightarrow 5 \rightarrow 1$, y la suma mínima es: $2 + 3 + 5 + 1 = 11$

Matriz M_2 : 15. El camino es $3 \rightarrow 9 \rightarrow 1 \rightarrow 2$, que suma: $3 + 9 + 1 + 2 = 15$.

Resolver el problema, según los siguientes pasos: 1) Escribir la ecuación en recurrencias que representa el cálculo de la suma mínima. 2) Diseñar un algoritmo de programación dinámica que resuelva el problema del cálculo de la suma mínima en una matriz dada. 3) Analizar la eficiencia de la solución propuesta. No hace falta un análisis formal, basta con una idea genérica del coste. 4) Proponer un algoritmo que implemente una solución eficiente para resolver el problema.

Ej. 23 — Dadas tres cadenas de caracteres $c1, c2$, y $c3$ queremos ver si es posible obtener $c3$ intercalando los caracteres de las otras dos (sin cambiar el orden).

El intercalado de dos cadenas $c1$ y $c2$ es una configuración donde ambas se dividen en n y m subcadenas, tales que:

- $c1 = c1_1 + c1_2 + \dots + c1_n$
- $c2 = c2_1 + c2_2 + \dots + c2_m$
- $|n - m| \leq 1$

El **intercalado** es $c1_1 + c2_1 + c1_2 + c2_2 + \dots$, o bien $c2_1 + c1_1 + c2_2 + c1_2 + \dots$, donde $a + b$ representa la operación de concatenación de esas dos cadenas.

Ejemplos:

- Si $c1 = "aabcc"$, $c2 = "dbbca"$, $c3 = "aadbcbcac"$

Respuesta: sí. $c1 = "aa" + "bc" + "c"$, y $c2 = "dbbc" + "a"$.

Intercalando las dos particiones de las cadenas tenemos:

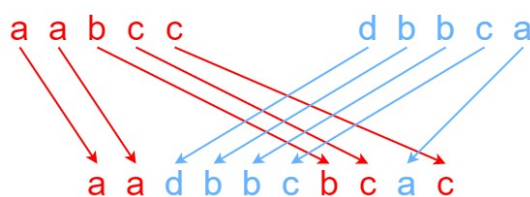
$"aa" + "dbbc" + "bc" + "a" + "c" = "aadbcbcac"$.

- Si $c1 = "aabcc"$, $c2 = "dbbca"$,
 $c3 = "aadbbaacc"$

Respuesta: no.

- Si $c1 = ""$, $c2 = ""$, $c3 = ""$

Respuesta: sí.



Resolver el problema, según los siguientes pasos: 1) Escribir la ecuación en recurrencias que representa el cálculo de la suma mínima. 2) Diseñar un algoritmo recursivo de programación dinámica que resuelva el problema del cálculo de la suma mínima en una matriz dada siguiendo la ecuación en recurrencias. 3) Analizar la eficiencia de la solución propuesta. No hace falta un análisis formal, basta con una idea genérica del coste. 4) Proponer un algoritmo que implemente una solución eficiente para resolver el problema.